

When AI Designs AI: Innovation or Imitation?

Yikang Yang^{1,2,4} Zhengxin Yang^{1,2,3*} Luzhou Peng^{2,4} Minghao Luo⁵
Yanqi Kan^{2,4} Wanling Gao^{2,3} Jianfeng Zhan^{2,3}

State Key Lab of Processors, Institute of Computing Technology, CAS, Beijing 100190, China¹

Institute of Computing Technology, Chinese Academy of Sciences²

BenchCouncil (International Open Benchmark Council)³

University of Chinese Academy of Sciences⁴

Department of Computer Science McCormick School of Engineering Northwestern University Evanston, IL, USA⁵

{yangyikang23s, yangzhengxin}@ict.ac.cn

Abstract

Recent advances in LLM agents have made them increasingly capable of designing methods for complex AI tasks. This raises two central questions about agent-designed methods relative to human-designed methods: how well they perform, and how different their algorithmic designs are. To study these questions, this paper introduces an analysis that derives task-specific algorithmic design spaces from human-designed methods, maps both human- and agent-designed methods into these spaces, and quantifies their algorithmic differences at the module level. Widely used LLM agents are evaluated on a suite of representative, open-ended AI tasks spanning multiple modalities, and the methods they design are analyzed in terms of both task performance and algorithmic differences from human-designed methods. Experimental results show that current agents can occasionally match or surpass human state-of-the-art (SOTA) performance (10/72 configurations), but such success does not generalize reliably across tasks or agents. Moreover, 96.8% of agent-designed methods fall within human-derived algorithmic design spaces, largely recombining algorithmic choices found in human-designed methods, while nearly half exactly match an existing human algorithmic design. Taken together, these findings suggest that although current agents can occasionally match or surpass human SOTA performance, their algorithmic designs remain within human-derived algorithmic design spaces, reflecting the reuse and recombination of algorithmic choices.

1. Introduction

LLM agents extend chat-only language models into systems capable of using tools and interacting with external environments [23, 25, 26, 37]. These advances have enabled LLM agents to tackle increasingly complex tasks across software engineering, web interaction, scientific discovery, and other domains [12, 17, 31, 36, 44]. These capabilities naturally lead to the idea of using LLM agents to design methods for complex AI tasks. This raises two central questions about agent-designed methods relative to human-designed

methods: *how well they perform, and how different their algorithmic designs are.*

Existing work has developed benchmarks for evaluating LLM agents across a range of AI research and engineering tasks [3, 14, 18, 19, 33]. Most of these benchmarks primarily assess agents based on final task performance, showing whether they achieve competitive results but offering limited insight into the algorithmic choices behind their performance. InnoGym is a recent exception that evaluates methodological novelty, but its reliance on LLM-as-a-judge introduces model-dependent bias [40]. In addition, some benchmark tasks offer limited room for open-ended algorithmic exploration, either because their solution paths are constrained or because performance is already close to saturation [3, 14, 18, 19]. These limitations make existing benchmarks insufficient on their own for analyzing the algorithmic choices of agent-designed methods, and their relationship to performance.

To address these limitations, this paper introduces a suite of representative, open-ended AI tasks that span multiple modalities and offer meaningful room for further performance improvement. Each task includes a human leaderboard with corresponding reference papers and code. This paper develops a human-in-the-loop workflow for building task-specific algorithmic design spaces from the referenced methods. Both human and agent-designed methods are then represented as coordinates in the corresponding task-specific design space. This common representation enables joint analysis of where agents explore, how their methods differ from human designs, and how these choices relate to performance.

This study evaluates widely used LLM agents on the task suite. Current agents can occasionally reach or surpass Human SOTA, but do so in only 10 of the 72 evaluation configurations and on only 3 of the 6 tasks. Meanwhile, most agent-generated methods can be explained as recombinations or local variations of human methods: nearly half algorithmically match an existing human method, and over 70% differ from the nearest human method in at most one module. This study also identifies several recurring patterns in agent research, including limited use of external knowledge, limited exploration of algorithmic design spaces, and a strong preference for ensemble prediction. These find-

*Zhengxin Yang is the corresponding author

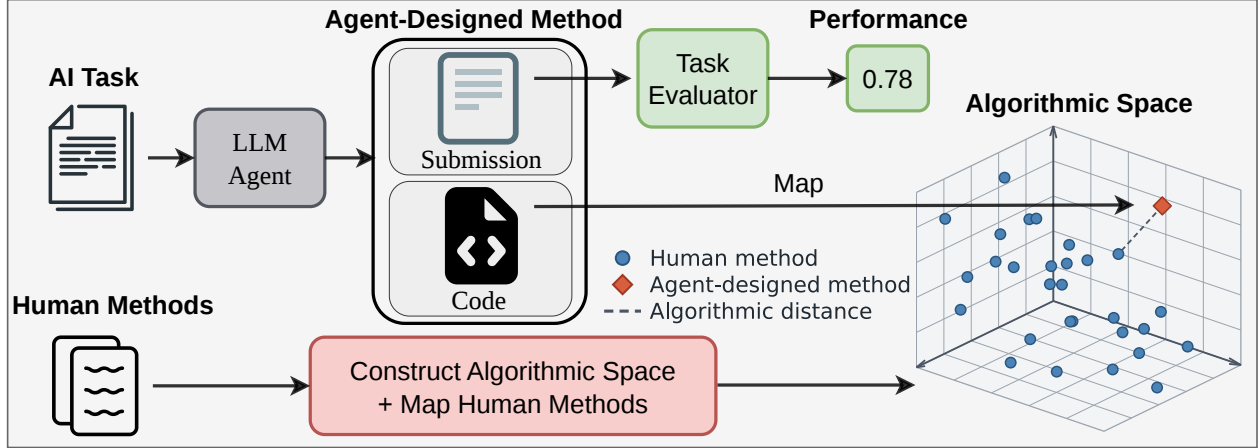


Figure 1. Overview of the analysis methodology. An LLM agent develops and executes code for an AI task, producing submissions that are evaluated by a task-specific evaluator to measure performance. The code is mapped into an algorithmic design space constructed from human reference methods. Human- and agent-designed methods are represented in the same task-specific space, enabling their algorithmic differences to be quantified at the module level.

ings suggest that current agents can occasionally reach the human frontier, but their exploration remains centered on established human methods and covers only a limited range of algorithmic designs.

The main contributions of this paper are as follows:

- This paper provides a systematic empirical study of how widely used LLM agents design methods for complex AI tasks, analyzing method performance, algorithmic designs, and patterns in agent research.
- To support this study, this paper introduces a suite of representative, open-ended AI tasks spanning multiple modalities, together with human leaderboards and corresponding reference papers and code.
- This paper develops a human-in-the-loop workflow that derives task-specific algorithmic design spaces from human-designed methods and represents both human- and agent-designed methods within them.

The remainder of this paper is organized as follows. Section 2 reviews related work on LLM agents and benchmarks for AI tasks. Section 3 describes the analysis methodology of this paper. Section 4 presents the experimental results on agent performance, algorithmic designs, and patterns in agent research. Section 5 discusses the implications of these findings and possible routes toward more reliable progress. Section 6 concludes the paper.

2. Related Work

2.1. LLM Agents

LLM agents extend language models into systems that can observe external environments, invoke tools, and take actions [30, 35]. In early work, ReAct developed a mechanism for LLMs to interact with environments [37], while

Toolformer focused on enabling language models to use tools [25]. Subsequent work enhanced agents’ capabilities in feedback-based self-reflection, long-term memory management, and reusable skill accumulation [23, 26, 29]. Today, Claude Code [1], Codex [21], and Gemini CLI [11] are among the most widely used coding agents. Academic work has also explored specialized systems for AI research, including AIDE [15] and MLEvolve [7], with related efforts using LLMs to iteratively search for and improve candidate programs [20, 24]. Rather than proposing a new agent system, this paper evaluates widely used agents on AI tasks and analyzes the performance and algorithmic designs of the methods they produce.

2.2. Benchmarks for LLM Agents on AI Tasks

Existing benchmarks evaluate LLM agents on machine learning experimentation and engineering, AI research challenges, paper replication, and frontier AI R&D [3, 9, 14, 18, 19, 27, 33, 41]. Most assess agents primarily through task completion and final performance, offering limited insight into the algorithmic choices behind the resulting methods. Some also include tasks whose solution paths are constrained or whose performance is close to saturation, limiting open-ended algorithmic exploration [3, 14, 19]. InnoGym is a recent exception that evaluates both performance gain and methodological novelty [40]. Its novelty evaluation, however, relies on LLM-as-a-judge comparisons, which can introduce model-dependent bias and inconsistency. Rather than evaluating novelty, this paper maps human and agent-designed methods into explicit task-specific algorithmic design spaces and measures the coordinate distances between them, making method differences traceable and auditable while partially mitigating subjectivity.

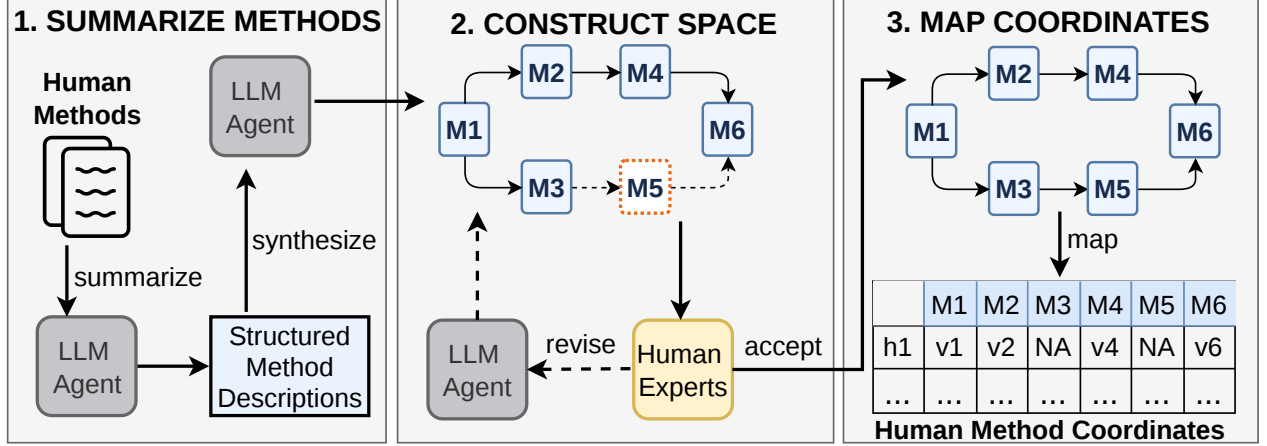


Figure 2. Human-in-the-loop construction of a task-specific algorithmic design space. LLM agents summarize human methods into structured method descriptions and use these descriptions to produce an initial version of the task-specific algorithmic design space. Human experts review the proposed modules, values, and dependencies, and LLM agents revise the design space based on their feedback until the revised design space is accepted. Each human method is then mapped to an algorithmic coordinate in the accepted design space.

3. Analysis Methodology

This section presents the methodology for analyzing methods designed by LLM agents in terms of both task performance and algorithmic design. As illustrated in Figure 1, an agent solves an AI task by developing and executing code that produces submissions. The submissions are evaluated using a task-specific performance metric, while the code is used to map the corresponding method into a task-specific algorithmic design space constructed from human reference methods.

3.1. Algorithmic Representation

Performance metrics show how well an agent-designed method performs, but reveal little about its underlying design choices or how they compare with those of human-designed methods. Direct comparisons based on textual similarity are unsuitable, since equivalent algorithms may be implemented differently, while methods with distinct core designs may share substantial surface-level similarities. A meaningful comparison therefore requires a common representation that abstracts away implementation-specific details while preserving core algorithmic choices. To this end, this paper represents human-designed and agent-designed methods within task-specific algorithmic design spaces, making these heterogeneous methods directly comparable.

For a task t , its algorithmic design space is represented as:

$$\mathcal{S}_t = (G_t, \{\mathcal{O}_{t,v}\}_{v \in V_t}), \quad (1)$$

where $G_t = (V_t, E_t)$ is a directed acyclic graph describing the main data flow of methods for the task. Each node $v \in V_t$ represents an algorithmic module, each edge represents a dependency between modules, and $\mathcal{O}_{t,v}$ contains the recorded

design choices for module v .

A method m is represented by an algorithmic coordinate

$$\mathbf{x}_t(m) = (x_{t,v}(m))_{v \in V_t}, \quad (2)$$

where $x_{t,v}(m) \in \mathcal{O}_{t,v}$ specifies its choice at module v . The coordinate captures algorithm-level choices while abstracting away implementation details such as hyperparameters and minor code-level differences.

For two methods represented in the same design space, their algorithmic distance is defined as the Hamming distance between their coordinates:

$$d_t(m_1, m_2) = \sum_{v \in V_t} \mathbb{I}[x_{t,v}(m_1) \neq x_{t,v}(m_2)]. \quad (3)$$

This distance counts the number of modules in which the two methods make different algorithmic choices.

Let $\mathcal{H}_t$ denote the collected human reference methods for task t . To position an agent-designed method a relative to existing human designs, this paper defines its distance to the human reference set as

$$d_t(a, \mathcal{H}_t) = \min_{h \in \mathcal{H}_t} d_t(a, h). \quad (4)$$

A distance of zero indicates that the agent-designed method shares an algorithmic coordinate with at least one collected human method, whereas larger values indicate differences across more modules. Because each term in the distance corresponds to a specific module, the comparison reveals not only how many algorithmic choices differ, but also where these differences occur.

3.2. Design Space Construction

This subsection describes how task-specific algorithmic design spaces are constructed from collected human reference

Task	Domain	Dataset	Metric	SOTA year	References
Image classification	Vision	CUB-200-2011 [28]	Accuracy $\uparrow$	2023	44
Emotion classification	NLP	GoEmotions [6]	Macro-F1 $\uparrow$	2026	24
Node classification	Graph	ogbn-arxiv [13]	Accuracy $\uparrow$	2023	81
Link prediction	Graph	ogbl-ppa [13]	Hits@100 $\uparrow$	2025	27
Time-series forecasting	Time series	ETTh1 [43]	MSE $\downarrow$	2026	76
Time-series forecasting	Time series	Weather [34]	MSE $\downarrow$	2025	75

Table 1. Overview of the task collection.

methods and subsequently used to represent both human- and agent-designed methods.

Figure 2 illustrates this human-in-the-loop construction process. LLM agents first convert each human reference method into a structured description of its algorithmic design, using the corresponding code as the primary source of evidence. Next, the LLM agents compare these descriptions across methods and group operations that serve the same role into shared modules. Within each module, alternative operations are recorded as candidate values, while dependencies between modules are inferred from the data flow of the corresponding methods.

Human experts with relevant domain knowledge then review the extracted data flows and the proposed organization of the design space from a global perspective. They adjudicate unresolved structural choices and provide corrective feedback when the framework does not adequately capture the collected human methods. The LLM agents revise the design space based on these decisions, and the revised version is reviewed again until it is accepted by the human experts.

Once the design space is accepted, LLM agents assign each human reference method an algorithmic coordinate and map each agent-designed method into the same space using the finalized module definitions and dependencies. Human experts then inspect a sample of the mappings for agent-designed methods to identify systematic assignment errors and apply the corresponding corrections consistently across the affected mappings, using the executed code as the primary evidence. When an agent-designed method introduces a choice not observed in the human references but compatible with an existing module, that choice is added as a new value. If the method cannot be expressed by the existing module structure, it is classified as *out of space* (OOS).

The resulting spaces are not intended to reproduce every implementation detail or provide fully precise descriptions of individual algorithms. Instead, they are designed to capture the core algorithm-level choices relevant to a shared and auditable comparison, allowing differences between human- and agent-designed methods to be traced to specific modules.

3.3. Task Collection

This subsection introduces the task collection used to study how LLM agents design methods for complex AI tasks. It presents the task selection principles and summarizes the resulting suite.

Task Design Principles. The task collection follows four design principles. First, the collection should span representative AI tasks across multiple domains and modalities. Second, each task should be open-ended, admitting multiple plausible algorithmic approaches rather than prescribing a single solution path. Third, each task should leave meaningful room for improvement over existing methods. Operationally, only tasks with a human SOTA method published between 2022 and 2026 are considered. Fourth, each task should have sufficient published human methods to support a manually curated human leaderboard, with corresponding papers and available code collected for its entries. The curated leaderboard provides a human performance baseline, while the collected references support the construction of task-specific algorithmic design spaces.

Task Suite. The resulting suite contains six tasks spanning computer vision, natural language processing, graph learning, and time-series forecasting. Each task is packaged as a self-contained project containing a task description, prepared data, an example submission, an evaluation script, a human leaderboard, and corresponding reference papers and available code. Across the six tasks, the collection contains 327 human reference entries. Table 1 summarizes the task suite, where SOTA year denotes the publication year of the top-ranked method on the corresponding human leaderboard [4, 8, 16, 32, 38, 42], and References denotes the number of collected reference entries.

4. Experiments

This section empirically evaluates widely used LLM agents as designers of methods for complex AI tasks. It asks three questions: how closely agent-designed methods approach the human frontier, how their algorithmic designs differ from existing human methods, and what characterizes the methods that reach that frontier. After describing the experimental setup, the analysis addresses these questions through task-level performance, algorithmic distance, and a focused

Agent	Ref.	Performance on Each Task						Overall
		CUB200 $\uparrow$	GoEmotions $\uparrow$	ogbn-arxiv $\uparrow$	ogbl-ppa $\uparrow$	ETTh1 $\downarrow$	Weather $\downarrow$	Rank
Human SOTA	–	0.9310	0.5518	0.7803	0.7655	0.3748	0.2170	2.67
CC-Opus	w/o	0.9355	0.5553	0.7458	0.6488	0.5047	0.2518	5.17
CC-Opus	w/	0.9241	0.5635	0.7709	0.6288	0.5266	0.2436	5.00
Codex	w/o	0.8234	0.5304	0.7431	0.1824	0.5281	0.2296	10.00
Codex	w/	0.9089	0.5411	0.7413	0.5049	0.5437	0.2399	9.83
Gemini CLI	w/o	0.9187	<u>0.5561</u>	0.7307	0.5530	0.5137	0.2326	7.00
Gemini CLI	w/	0.9268	<u>0.5629</u>	0.7686	0.6208	0.5297	0.2267	4.83
CC-DeepSeek	w/o	0.9172	0.5510	0.7308	0.5665	0.5154	0.2405	8.33
CC-DeepSeek	w/	0.8963	<u>0.5633</u>	0.7656	0.4947	0.5068	0.2321	6.33
CC-GLM	w/o	0.9299	<u>0.5596</u>	0.7467	0.5008	0.5119	0.2227	4.92
CC-GLM	w/	0.9299	<u>0.5557</u>	0.7657	0.5124	0.5271	0.2070	4.92
MLEvolve	w/o	0.9237	<u>0.5545</u>	0.6952	0.1962	0.7021	0.2347	10.00
MLEvolve	w/	0.9077	0.5417	0.7148	0.0966	0.6829	0.2797	12.00

Table 2. Best performance achieved within the evaluation budget for each agent, task, and reference condition. The Ref. column indicates whether prepared reference papers and corresponding code are provided (w/) or withheld (w/o). CC-Opus, CC-DeepSeek, and CC-GLM denote Claude Code using Claude Opus 4.8, DeepSeek 4 Pro, and GLM 5.2, respectively. Codex and MLEvolve use GPT 5.5, while Gemini CLI uses Gemini Flash 3.5. The Human SOTA row reports the best collected human result for each task. Upward and downward arrows indicate whether higher or lower values are better. Rank is the mean task-wise rank across Human SOTA and the twelve agent configurations. **Bold** values indicate the best result in each column, while underlined agent results meet or exceed Human SOTA.

examination of frontier-reaching methods. Finally, as a complementary analysis, the section examines whether prepared human references improve agent performance and whether agents actively seek and use external knowledge.

4.1. Experimental Setup

The evaluation comprises 72 experimental configurations, covering all $6 \times 6 \times 2$ combinations of (agent, task, reference condition). In each configuration, an agent receives the task description, prepared data, an example submission, and an evaluation interface, develops and executes an end-to-end method, and submits its predictions for evaluation. The evaluator returns only a scalar performance score without exposing individual test labels or targets. Each evaluated method is recorded as a solution, with a complete snapshot of its code, runnable script, submission file, method summary, and execution logs. Beyond task-specific performance, these methods are mapped into the corresponding algorithmic design spaces, while available agent traces are used to examine how agents conduct research. The evaluated agents and remaining experimental settings are described below.

Agents. Three Claude Code agents [1] use Claude Opus 4.8 [2], DeepSeek 4 Pro [5], and GLM 5.2 [39]; Codex [21] uses GPT 5.5 [22]; Gemini CLI [11] uses Gemini Flash 3.5 [10]; and MLEvolve [7] uses GPT 5.5.

Tasks. The six tasks are those introduced in Section 3.3. For brevity, CUB-200-2011 is referred to as CUB200.

Reference Conditions. Each (agent, task) pair is evaluated both with and without prepared reference papers and corresponding code. Internet access and all other available

tools remain enabled under both conditions.

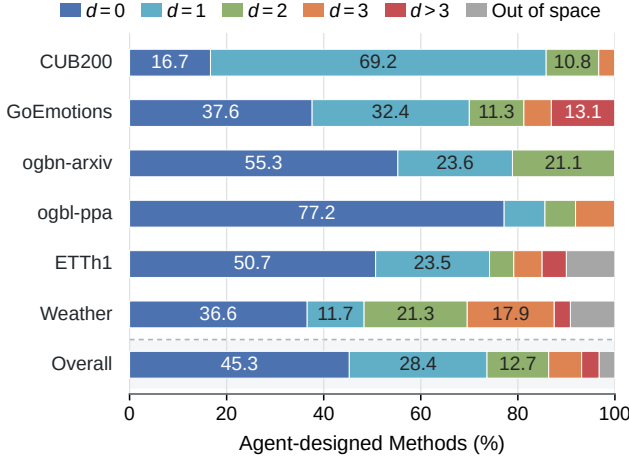
Execution Budget. Each experimental configuration is allocated up to 24 hours on a single NVIDIA V100 GPU. Within this budget, the agent may iteratively produce and evaluate up to 10 successive solutions.

4.2. Performance

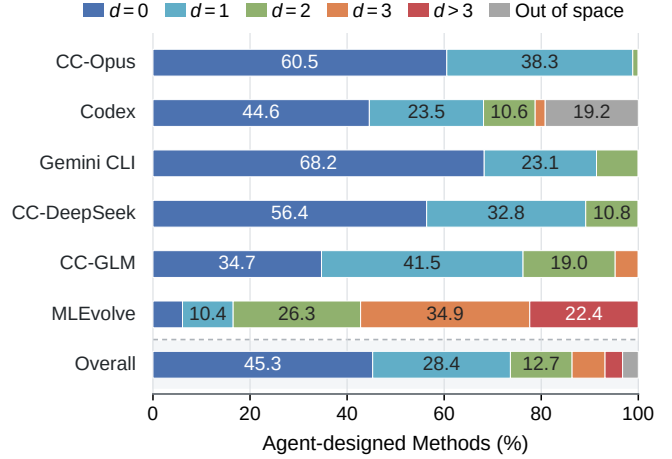
This subsection examines how closely agent-designed methods approach the human performance frontier across agents and tasks. Table 2 reports the best performance achieved in each experimental configuration and summarizes cross-task performance using average rank. Two main findings emerge from this comparison.

Current agents occasionally reach the human frontier, but such success does not generalize reliably. Among the 72 experimental configurations, 10 reach or surpass Human SOTA, but these successes occur on only 3 of the 6 tasks. Eight are concentrated on GoEmotions, while CUB200 and Weather each account for only one. Moreover, no agent under either reference condition reaches Human SOTA on more than two tasks. Human SOTA retains the best average rank at 2.67, leaving a clear gap to the strongest individual agent configuration, Gemini CLI with prepared references, at 4.83. When each agent’s ranks are averaged across the two reference conditions, the overall ordering is CC-GLM, CC-Opus, Gemini CLI, CC-DeepSeek, Codex, and MLEvolve.

The gap to the human frontier varies substantially across tasks. Agents perform most strongly on GoEmotions, where 8 of the 12 configurations surpass Human SOTA. CUB200 and Weather each exhibit only one frontier-



(a) Algorithmic distance distributions by task.



(b) Algorithmic distance distributions by agent.

Figure 3. Distributions of the minimum algorithmic distance from agent-designed methods to collected human methods, group separately by agent and task.

reaching result, while all configurations on ogbn-arxiv remain below but relatively close to Human SOTA. The largest gaps occur on ETTh1 and ogbl-ppa, where the best agent results still have 34.7% higher MSE and 15.2% lower Hits@100 than Human SOTA, respectively. These results show that the ability of current agents to reach the human frontier remains strongly task-dependent.

4.3. Algorithmic Exploration

This subsection examines agent-designed methods from two complementary perspectives: their positions relative to existing human methods and the breadth of their exploration within the task-specific algorithmic design spaces.

Agent-designed methods generally remain close to existing human methods. Figures 3a and 3b report the algorithmic distance distributions by task and agent, respectively. Across all agents and tasks, 45.3% of the methods share an algorithmic coordinate with at least one collected human method, while another 28.4% differ from the nearest human method in exactly one module. Thus, 73.7% lie within $d \leq 1$ of a human method. This proximity is observed across most tasks: five of the six tasks have at least 70% of their methods within $d \leq 1$, with Weather as the main exception at 48.3%. The distributions also vary across agents. For the five agents other than MLEvolve, the proportion of methods within $d \leq 1$ ranges from 68.1% to 98.8%. MLEvolve is a clear exception, with only 16.4% within $d \leq 1$ and 57.2% differing from the nearest human method in at least three modules.

Most agent-designed methods are recombinations or local variations of human designs. Among methods represented within the design spaces, 95.3% of module-level choices use values already observed in collected human

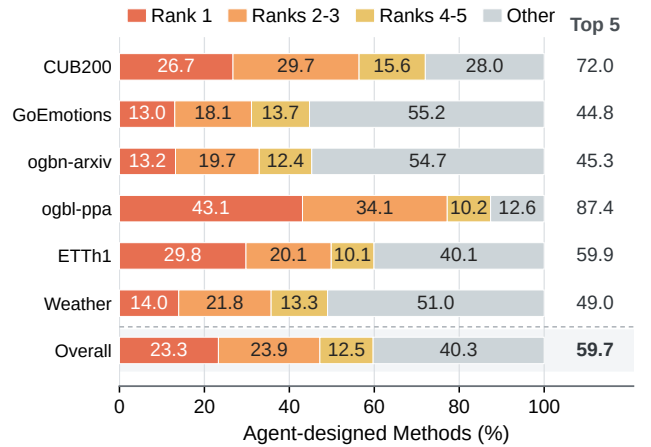


Figure 4. Concentration of agent-designed methods on the most frequently visited algorithmic coordinates within each task. Rank 1 denotes the most frequent coordinate, while Ranks 2–3 and Ranks 4–5 denote the corresponding groups of coordinates. Other includes less frequent coordinates and methods outside the constructed design spaces. Overall reports the mean distribution across the six tasks.

methods, while only 4.7% require previously unobserved values. Moreover, only 3.2% of the methods fall outside the constructed design spaces. Together with the distance distributions, these results indicate that agent-designed methods primarily reproduce or recombine established human choices.

Agents repeatedly explore a limited set of algorithmic coordinates. Figure 4 reports the proportions of agent-designed methods assigned to the five most frequent algorithmic coordinates in each task. Although each experimental

configuration can produce up to ten successive solutions, these solutions occupy a median of only 3.5 distinct coordinates. The five most frequent coordinates account for more than half of all agent-designed methods in four of the six tasks and 59.7% on average. These results indicate that agents repeatedly return to a small set of algorithmic designs rather than broadly covering the constructed design spaces.

4.4. Frontier-Reaching Methods

This subsection analyzes how high-performing agent-designed methods relate algorithmically to existing human methods.

High-performing methods generally remain close to existing human designs. Figure 5 compares the performance of agent-designed methods across algorithmic distance categories. None of the 23 solutions outside the constructed design spaces ranks in the top quartile for its task. Moreover, in five of the six tasks, the best-performing agent-designed method lies within $d \leq 1$. Weather is the only exception, with its best-performing agent-designed method located at $d = 3$.

Methods reaching Human SOTA generally recombine human-observed choices and rely on ensemble prediction. Among the 10 configurations that reach or surpass Human SOTA, 8 of them produce a SOTA-reaching method at $d \leq 2$, and none produces one outside the constructed design spaces. Moreover, 88.0% of the module-level choices in their best methods use values already observed in collected human methods. Notably, the best method in each of the 10 SOTA-reaching configurations uses ensemble prediction.

4.5. Use of Human Knowledge

This subsection examines whether providing agents with prepared human references helps them design methods for these tasks and whether agents actively seek and use existing human knowledge.

Prepared references do not consistently improve performance. Table 2 provides 36 matched comparisons between experimental configurations that differ only in whether prepared reference papers and corresponding code are provided. Providing references improves performance in 20 comparisons, reduces it in 15, and leaves it unchanged in 1. Moreover, among the 10 configurations that reach or surpass Human SOTA, 5 use prepared references and the other 5 do not. Thus, under the experimental setup of this study, providing prepared references does not yield a clear improvement in task performance or increase the number of configurations that reach or surpass Human SOTA.

Agents make limited use of external knowledge. Among the agents studied, MLEvolve is the only one without the ability to actively access the Internet; all other agents can search for relevant online resources, and Internet use is unrestricted in the experiments. Nevertheless, only 2 of the 60 configurations involving these five agents conduct task-

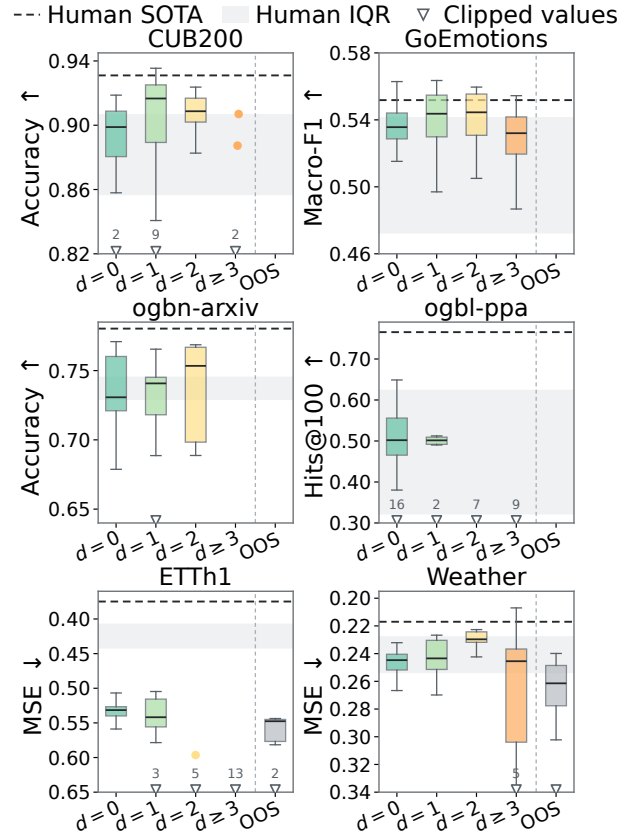


Figure 5. Task performance across algorithmic distance categories. Boxes show the performance distributions for categories containing at least eight solutions, while individual points are shown for smaller categories. Dashed lines indicate Human SOTA, while the regions labeled Human IQR span the 25th to 75th percentiles of the collected human performance results. OOS denotes solutions outside the constructed design spaces. Open triangles indicate results beyond the displayed vertical range, with numbers reporting their counts when greater than one.

specific online searches for existing methods. Even when references are provided, agents access only 5.4% of them and consult a top-3 reference for the corresponding task in only 8 of the 36 configurations. These results suggest that agents rely primarily on knowledge learned during LLM pre-training rather than systematically surveying existing human methods.

5. Discussion

Most methods produced by current agents can be understood as recombinations of algorithmic choices observed in human methods. Methods that reach or surpass Human SOTA also remain predominantly close to existing human designs, although Weather shows that larger departures can occasionally succeed. These successes, however, remain

task-dependent and coexist with limited exploration of the algorithmic design spaces and limited use of external knowledge. Taken together, these findings indicate that current agents can occasionally produce strong methods but cannot yet do so reliably across tasks.

These findings raise a broader question: how can agents turn occasional successes into reliable progress? From the perspective of algorithmic design spaces, such progress may follow three broad routes, depending on whether it preserves an existing coordinate, moves to another coordinate, or expands beyond the existing space: (1) **coordinate-preserving optimization**, which adjusts parameters or implementation details while keeping the coordinate fixed; (2) **within-space recombination**, which moves to a new coordinate primarily by combining established module choices; (3) **space expansion**, which seeks designs that substantially extend or fall outside the constructed space.

Coordinate-preserving optimization is the most accessible, but its performance ceiling remains bounded by the selected coordinate. Space expansion offers the highest theoretical ceiling because it is not constrained by the existing design space, but it is also the most difficult route to pursue reliably. Within-space recombination offers a trade-off, with a higher performance ceiling than coordinate-preserving optimization and greater feasibility than space expansion. The experimental results in this paper also demonstrate the feasibility of this route, as most methods that reach or surpass Human SOTA appear to follow it.

6. Conclusion

This paper investigates how well current LLM agents can design methods for complex AI tasks and how their algorithmic designs relate to existing human methods. To support this investigation, an analytical perspective based on algorithmic design spaces is introduced, through which human- and agent-designed methods are mapped into shared task-specific spaces for module-level comparison. Widely used LLM agents are evaluated on a suite of representative, open-ended AI tasks spanning multiple modalities. The results show that current agents can occasionally reach or surpass Human SOTA, but such success remains task-dependent and unreliable, while most agent-designed methods primarily recombine algorithmic choices observed in human methods. Finally, this paper outlines three possible routes toward more reliable progress: coordinate-preserving optimization, within-space recombination, and space expansion.

References

- [1] Anthropic. Claude code by anthropic — ai coding agent, terminal, ide, 2026. <https://claude.com/claude-code>. 2, 5
- [2] Anthropic. Introducing claude opus 4.8 anthropic, 2026. <https://www.anthropic.com/news/claude-opus-4-8>. 5
- [3] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. 1, 2
- [4] Po-Yung Chou, Yu-Yung Kao, and Cheng-Hung Lin. Fine-grained visual classification with high-temperature refinement and background suppression. *ArXiv*, abs/2303.06442, 2023. 4
- [5] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. *ArXiv*, abs/2606.19348, 2026. 5
- [6] Dorottya Demszky, Dana Movshovitz-Attias, Jeongwoo Ko, Alan Cowen, Gaurav Nemade, and Sujith Ravi. GoEmotions: A dataset of fine-grained emotions. In *The 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. 4
- [7] Shangheng Du, Xiangchao Yan, Jinxin Shi, Zongsheng Cao, Shiyang Feng, Zichen Liang, Boyuan Sun, Tianshuo Peng, Yifan Zhou, Xin Li, Jie Zhou, Liang He, Bo Zhang, and Lei Bai. Mlevolve: A self-evolving framework for automated machine learning algorithm discovery. *ArXiv*, abs/2606.06473, 2026. 2, 5
- [8] Keyu Duan, Qian Liu, Tat-Seng Chua, Shuicheng Yan, Wei Tsang Ooi, Qizhe Xie, and Junxian He. Simteg: A frustratingly simple approach improves textual graph learning. *ArXiv*, abs/2308.02565, 2023. 4
- [9] Nicholas Edwards, Yukyung Lee, Yujun Audrey Mao, Yulu Qin, Sebastian Schuster, and Najoung Kim. Rexbench: Can coding agents autonomously implement AI research extensions? In *The 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026. 2
- [10] Google. Gemini 3.5 flash — gemini api — google ai for developers, 2026. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-5/>. 5
- [11] Google. Build, debug & deploy with ai — gemini cli, 2026. <https://gemini-cli.com/>. 2, 5
- [12] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. 1
- [13] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *The Thirty-fourth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2020. 4
- [14] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentbench: Evaluating language agents on machine learning experimentation. In *The Forty-first International Conference on Machine Learning (ICML)*, 2024. 1, 2

- [15] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: ai-driven exploration in the space of code. *ArXiv*, abs/2502.13138, 2025. 2
- [16] Florian Lecourt, Madalina Croitoru, and Konstantin Todorov. Linguistic signatures for enhanced emotion detection. In *Proceedings of the ACM Web Conference 2026*, 2026. 4
- [17] Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards end-to-end automation of AI research. *Nature*, 651(8107):914–919, 2026. 1
- [18] Alisia Maria Lupidi, Bhavul Gauri, Thomas Foster, Bassel Al Omari, Despoina Magka, Alberto Pepe, Alexis Audran-Reiss, Muna Aghamelu, Nicolas Mario Baldwin, Lucia Cipolina-Kun, Jean-Christophe Gagnon-Audet, Chee Hau Leow, Sandra Lefdal, Hossam Mossalam, Abhinav Moudgil, Saba Nazir, Emanuel Tewolde, Isabel Urrego, Jordi Armengol-Estap , Amar Budhiraja, Gaurav Chaurasia, Abhishek Charnalia, Derek Dunfield, Karen Hambardzumyan, Daniel Izcovich, Martin Josifoski, Ishita Mediratta, Kelvin Niu, Parth Pathak, Michael Shvartsman, Edan Toledo, Anton Protopopov, Roberta Raileanu, Alexander H. Miller, Tatiana Shavrina, Jakob N. Foerster, and Yoram Bachrach. Airstbench: a suite of tasks for frontier AI research science agents. *ArXiv*, abs/2602.06855, 2026. 1, 2
- [19] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob N. Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. Mlgym: A new framework and benchmark for advancing AI research agents. *ArXiv*, abs/2502.14499, 2025. 1, 2
- [20] Alexander Novikov, Ng n V u, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav M. Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, Matej Balog, and Google DeepMind. Alphaevolve: A coding agent for scientific and algorithmic discovery. *ArXiv*, abs/2506.13131, 2025. 2
- [21] OpenAI. Codex in chatgpt — ai coding agents for software engineering — openai, 2026. <https://openai.com/codex/>. 2, 5
- [22] OpenAI. Introducing gpt-5.5 — openai, 2026. <https://openai.com/index/introducing-gpt-5-5/>. 5
- [23] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph Gonzalez. Memgpt: Towards llms as operating systems. *ArXiv*, abs/2310.08560, 2023. 1, 2
- [24] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. 2
- [25] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *The Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 1, 2
- [26] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *The Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 1, 2
- [27] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. Paperbench: Evaluating AI’s ability to replicate AI research. In *The Forty-second International Conference on Machine Learning (ICML)*, 2025. 2
- [28] Catherine Wah, Steve Branson, Peter Welinder, Pietro Perona, and Serge J. Belongie. The caltech-ucsd birds-200-2011 dataset. 2011. 4
- [29] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi (Jim) Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *ArXiv*, abs/2305.16291, 2023. 2
- [30] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. 2
- [31] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muenighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. 1
- [32] Yihang Wang, Yuying Qiu, Peng Chen, Kai Zhao, Yang Shu, Zhongwen Rao, Lujia Pan, Bin Yang, and Chenjuan Guo. Towards a general time series forecasting model with unified representation and adaptive transfer. In *The Forty-second International Conference on Machine Learning (ICML)*, 2025. 4
- [33] Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Bradley, Lawrence Chan, Michael Chen, Joshua M Clymer, Jai Dhyani, Elena Elicheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-bench: Evaluating frontier AI r&d capabilities of language model agents against human experts. In *The Forty-second International Conference on Machine Learning (ICML)*, 2025. 1, 2
- [34] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with autocorrelation for long-term series forecasting. In *The Thirty-fifth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2021. 4

- [35] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. The rise and potential of large language model based agents: a survey. *Science China Information Sciences*, 68(2):121101, 2025. [2](#)
- [36] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024. [1](#)
- [37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. [1](#), [2](#)
- [38] Yuqi Yuan, Xiong Luo, Qiaojuan Peng, and Wenbing Zhao. Graft: infusing pre-trained transformers with relational structure for time series forecasting. In *The Fortieth AAAI Conference on Artificial Intelligence (AAAI)*, 2026. [4](#)
- [39] GLM-4.5 Team Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chengxing Xie, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Hui Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jin-Cheng Du, Jinzhu Wu, Kedong Wang, Lei Li, Linxi Jim Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xue Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuan Wen, Yukuo Cen, Yu Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqian Liu, Zixuan Li, Bo Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chen Huang, Chen Wei-Hua Hu, Chenhui Zhang, Chenzheng Zhu, Congfeng Yin, Daoyan Lin, Da-Wei Yang, Dingkun Wang, Ding Ai, Erle Zhu, Fang-Yu Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Hai Zhao, Haiyi Hu, Hanchen Zhang, Han Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hong Yan, Hongyu Ge, Huan Liu, Hua Chu, Jian Zhao, Jiachen Wang, Jiajing Zhao, Jiaming Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jia yin Zhao, Jijie Li, Jing An, Jing Li, Jing Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kai Zhou, Kang Wei, Kedong Wang, Keyu Luo, Laiqiang Zhang, Lei Sha, Liang Xu, Lindong Wu, Li Ding, Lu Chen, Minghao Li, Nianyi Lin, Pankova Ta, Qiang Zou, Rong-Huang Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuying Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiangtian Kuang, Xiang Cheng, Xiangyang Li, Xiao Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xi Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xunkai Zhang, Yandong Wu, Yanfu Li, Yangyi Fang, Yidong Wang, Yi fan Zhu, Yi-Shuai Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yi xin Geng, Yong Yan, Yonglin Tan, Yue Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yu Duan, Yuxuan Zhang, Ze-Xian Liu, Zhen Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuofu Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yu-Chao Dong, and Jie Tang. Glm-5: from vibe coding to agentic engineering. *ArXiv*, abs/2602.15763, 2026. [5](#)
- [40] Jintian Zhang, Kewei Xu, Jingsheng Zheng, Zhuoyun Yu, Yuqi Zhu, Yujie Luo, Lanning Wei, Shuofei Qiao, Lun Du, Da Zheng, Shumin Deng, Huajun Chen, and Ningyu Zhang. Innogym: Benchmarking the innovation potential of AI agents. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. [1](#), [2](#)
- [41] Yunxiang Zhang, Muhammad Khalifa, Shitanshu Bhushan, Grant D Murphy, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. MLRC-bench: Can language agents solve machine learning research challenges? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2026. [2](#)
- [42] Qifang Zhao, Weidong Ren, Tianyu Li, Hong Liu, Xingsheng He, and Xiaoxiao Xu. GraphGPT: Generative pre-trained graph eulerian transformer. In *The Forty-second International Conference on Machine Learning (ICML)*, 2025. [4](#)
- [43] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *The Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI)*, 2021. [4](#)
- [44] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. [1](#)